Computer Graphics 2025–2026
Report for Labs 6–8: Optimal Transport and Fluid Simulation

Alfreds Sarocinskis

Introduction

This is my second computer graphics project: a 2D optimal transport solver and a small fluid simulator built on top of it. The work is split over three labs. Lab 6 builds a Voronoi diagram of a set of points by clipping polygons. Lab 7 adds a weight to each point and optimises the weights so that every cell ends up with a chosen area (this is semi-discrete optimal transport). Lab 8 reuses all of that, plus one extra "air" weight, to simulate a block of liquid falling under gravity.

Everything happens inside the unit square $[0,1]^2$. The code is split into a few classes:

- `Vector`: a 2D point/vector with the usual arithmetic and a dot product.
- `Polygon`: an ordered list of vertices, with `area()` (shoelace formula), `centroid()` and `integral_square_distance()`.
- `VoronoiDiagram`: the points, their weights, and `compute()`, which clips each cell by all bisectors (and, in Lab 8, by a disk).
- `OptimalTransport`: runs the L-BFGS optimisation of the weights (`evaluate` gives the objective and its gradient).
- `Fluid`: the particles, their velocities, and `time_step()` / `run_simulation()`.

The vector class, the rasteriser `save_frame`, the SVG writer and the L-BFGS library (`lbfgs.c/h`) were given to us as a template. I wrote the polygon clipping, the area/centroid/integral functions, the optimal transport objective and gradient, the disk clipping and the whole fluid time step. It is all one file, compiled with OpenMP (the loop over cells runs in parallel):

```
g++ -std=c++17 -O2 -fopenmp -I. main.cpp lbfgs.c -o main
./main
```

All the images and timings below were made on an M4 MacBook Air.

1 Lab 6: Voronoi diagrams by polygon clipping

A Voronoi cell of a point P_i is the set of points that are closer to P_i than to any other point. It is the intersection of the half-planes given by the bisectors of the segments $[P_i, P_j]$, so I build it step by step: start with the unit square and clip it with one bisector at a time.

The clipping is the Sutherland–Hodgman algorithm (`clip_by_bisector`). I walk around the polygon edge by edge and mark each vertex as inside or outside the current half-plane; when an edge crosses the bisector I add the intersection point. For a plain Voronoi diagram the bisector goes through the midpoint $M = \frac{1}{2}(P_0 + P_i)$ with direction $\vec{d} = P_i - P_0$, and a point X is kept when $(X - M) \cdot \vec{d} < 0$, i.e. on the P_0 side. The crossing point on an edge $[A, B]$ is at

$$t = \frac{(M - A) \cdot \vec{d}}{(B - A) \cdot \vec{d}}.$$

`VoronoiDiagram::compute` just does the simple $O(N^2)$ version: each cell is clipped against every other point. It is easy to write, and an OpenMP `parallel for` over the cells makes it fast enough for the sizes I use here.

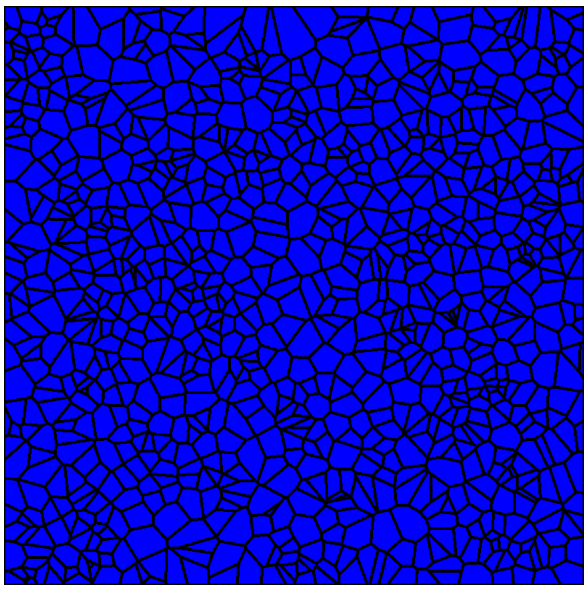


Figure 1: Lab 6: Voronoi diagram of $N = 1000$ random points. The cells have very different sizes because the points are placed randomly. This is exactly what optimal transport fixes in Lab 7.

2 Lab 7: Semi-discrete optimal transport

Lab 7 turns the question around. Instead of taking the cells as they come out, I look for weights w_i so that each cell of the resulting *Laguerre* (power) diagram has a chosen area. Here I want them all equal, so the target is $\lambda_i = 1/N$.

Laguerre cells. Giving each point a weight just shifts the dividing line away from the midpoint. I reuse the same clipping code; the only change is that the point M becomes

$$M = \frac{P_0 + P_i}{2} + \frac{w_0 - w_i}{2\,\|P_i - P_0\|^2}\,(P_i - P_0).$$

With equal weights this is back to the Voronoi midpoint, so one function covers both labs.

Objective and gradient. The optimal transport dual is concave, so the code minimises minus it,

$$g(W) = \sum_{i=1}^{N} \Big(\int_{\text{cell}_i} \|x - P_i\|^2\,dx \;-\; w_i \,\big(\text{area}(\text{cell}_i) - \lambda_i\big)\Big).$$

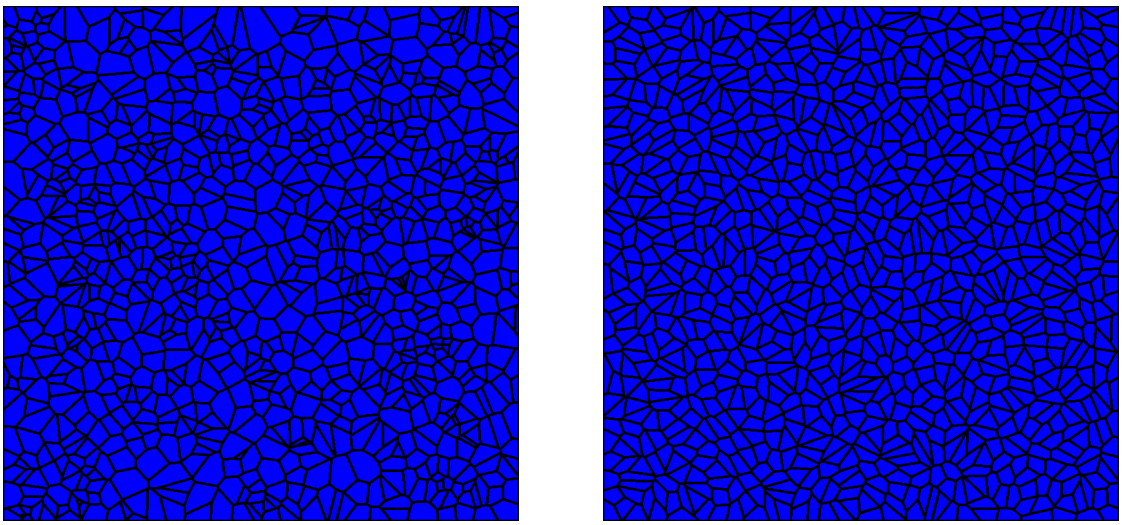
Its gradient in w_i is just

$$\frac{\partial g}{\partial w_i} = \text{area}(\text{cell}_i) - \lambda_i.$$

So at the optimum the gradient is zero and every cell has the target area $\lambda_i = 1/N$. The integral $\int_{\text{cell}} \|x - P_i\|^2 dx$ is computed in `integral_square_distance` by splitting the polygon into triangles from its first vertex and using the per-triangle formula

$$\int_T \|x - P_i\|^2\,dx = \frac{\text{area}(T)}{6} \sum_{1 \le k \le l \le 3} (c_k - P_i) \cdot (c_l - P_i),$$

where c_1, c_2, c_3 are the triangle's vertices. I pass the value and the gradient to L-BFGS (`liblbfgs`), which rebuilds the diagram on every call and converges in a few hundred iterations.



Before: Voronoi ($w_i = 0$) After: optimised Laguerre cells

Figure 2: Lab 7: the same $N = 1000$ points before and after optimal transport. The weights make all cells almost equal in area, and each cell still belongs to its point. The largest relative area error drops from about 357% to 0.3% (see Table 1).

3 Lab 8: Fluid simulation

Lab 8 uses the Gallouet–Mérigot scheme. The fluid is a set of particles. At each step I solve an optimal transport problem that gives every particle a cell of equal area (this stops the fluid from being compressed), and then I pull each particle towards the centroid of its cell.

Partial transport with air. The fluid only takes up a fraction $f = 0.45$ of the square, so I add one extra "air" weight w_{air} (the last entry of the weight vector) for the empty part. In `evaluate` the N_f fluid cells aim for area f/N_f and the air aims for $1 - f$; the gradient for the air weight is (estimated air area) $- (1 - f)$. Each fluid cell is also intersected with a disk of radius $\sqrt{w_i - w_{\text{air}}}$ around the particle, so a particle never takes fluid that is far away from it. The disk is drawn as a 50-sided polygon and clipped edge by edge with `clip_by_edge` (Sutherland–Hodgman again, this time against the half-plane of each disk edge, whose outward normal is $(\mathrm{d}y, -\mathrm{d}x)$).

Particle dynamics. Once the cells are known I move each particle with semi-implicit Euler, under gravity and a spring that pulls it to its cell centroid C_i (this spring is the incompressibility force):

$$\vec{F}_i = \frac{1}{\varepsilon^2}\,(C_i - X_i) + m\,\vec{g}, \qquad \vec{v}_i \mathrel{+}= \frac{\Delta t}{m}\,\vec{F}_i, \qquad X_i \mathrel{+}= \Delta t\,\vec{v}_i,$$

with $\varepsilon = 0.004$, $m = 200$, $\vec{g} = (0, -9.81)$ and $\Delta t = 0.002$. A particle that leaves the square is put back in and its normal velocity is flipped. I use $N = 300$ particles, started as a regular grid (with a bit of jitter) in a square block of area f near the top, which then falls and collects at the bottom.

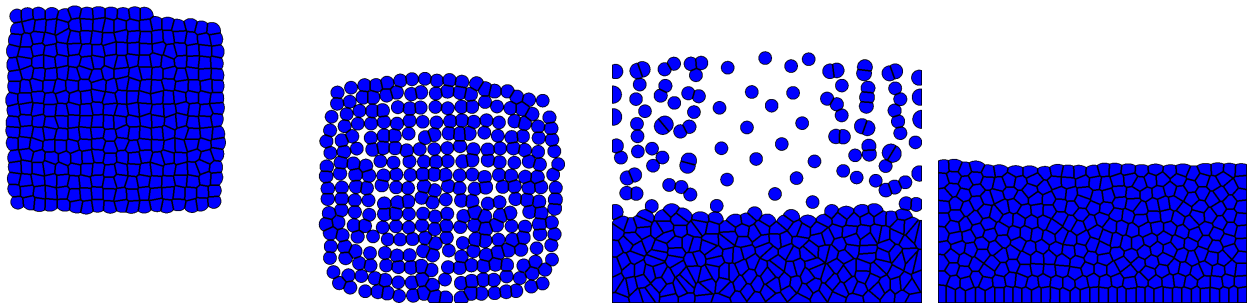


Figure 3: Lab 8: a block of fluid (300 particles) dropped from near the top. It falls, hits the floor and splashes, then settles into a pool with a flat surface. The total fluid area stays at $f = 0.45$ the whole time, so the optimal transport really does keep the liquid incompressible.

A bug I had to fix: keeping the volume. My first runs looked good for about a hundred frames and then suddenly blew up: one frame showed a few cells stretching across the whole square, and after that the fluid filled the entire domain instead of keeping its 0.45 area. The problem was a corner case in the disk clipping. When the optimiser briefly pushed the air weight above a particle's weight, the radius $\sqrt{w_i - w_{\text{air}}}$ became imaginary. I had set it to $R = 0$, but a zero-radius disk has edges with a zero normal, and my half-plane test then kept every point instead of clipping everything away. The cell grew to the full square, the optimiser saw an area of 1.0 where it wanted 0, pushed the air weight even higher, and the run diverged. The fix was one line: when $w_i - w_{\text{air}} \leq 0$ I just empty the cell (the particle has no fluid). After that the area stayed at 0.45 for the whole run.

Extra work beyond the mandatory part: viscosity and restitution. With only the required forces the fluid is perfectly elastic. The wall bounces keep all the energy and the spring gives back what it stores, so the liquid keeps jumping around and never settles (Figure 4, left). To make it look more like real water I added a bit of energy loss, behind a `use_damping` flag so the original behaviour is one boolean away. Each step I multiply every velocity by 0.997 (a simple viscosity), and wall bounces keep only 0.8 of the speed (restitution). With this on, the splash calms down and the fluid settles into a still pool (Figure 4, right).

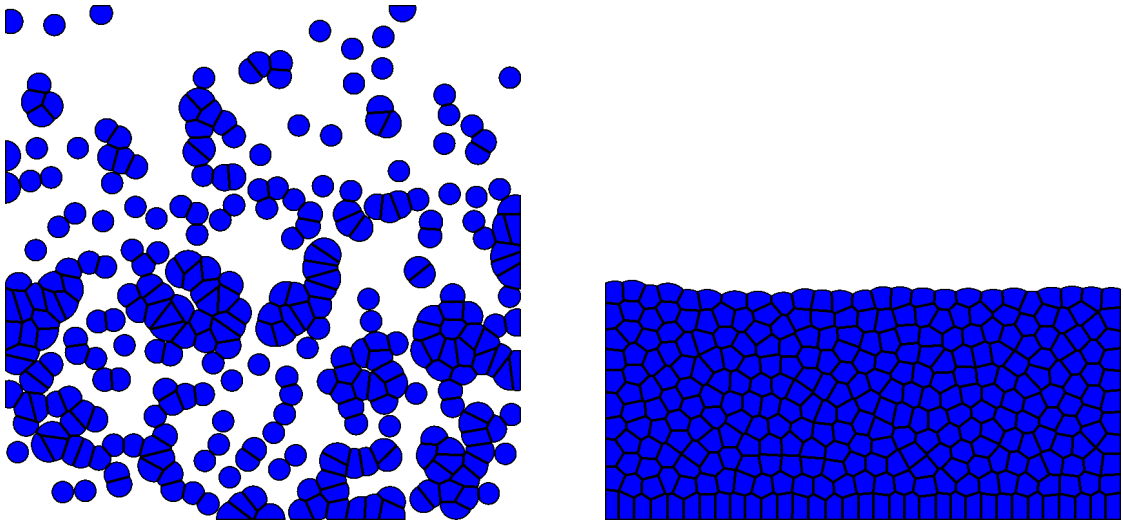


Figure 4: Late-time comparison. Left: the required elastic fluid is still scattered and bouncing after 1500 frames. Right: with viscosity ($\times 0.997$ per step) and softer bounces (restitution 0.8) the same fluid settles into a calm pool. The volume is the same in both; only the energy is different.

4 Timings

One optimal transport solve is the most expensive thing here (Lab 8 runs one per frame). The table sweeps the number of points N for a single solve with target area $1/N$ and seed 42. `maxRel` is the largest relative area error $\max_i |\text{area}_i - 1/N|/(1/N)$, shown for the plain Voronoi diagram and after optimisation.

N	Voronoi (ms)	OT solve (ms)	L-BFGS iters	Voronoi maxRel	OT maxRel
100	1.79	98.6	81	170%	0.022%
200	4.09	478.1	124	186%	0.053%
500	22.1	5 296	237	295%	0.103%
1000	89.0	32 964	361	357%	0.322%
2000	343.5	241 965	684	309%	0.427%

Table 1: One optimal transport solve for different N. The single Voronoi build is the $O(N^2)$ clipping; the OT solve is many such builds, one per L-BFGS call. Optimal transport cuts the worst area error by three to four orders of magnitude, from about 170 to 360% down to well under half a percent, so the cells become almost uniform. After optimisation the cell areas always add up to 1.0, so they cover the whole square.

Two things to note. First, one Voronoi/Laguerre build grows like N^2: going from $N = 100$ to $N = 2000$ (20× more points) makes the build about 190× slower, close to the 400× of a pure N^2 law but smaller because of OpenMP and fixed costs at small N. Second, the number of L-BFGS iterations grows slowly (from 81 to 684), so the solve time is basically the N^2 build cost times a slowly growing iteration count. That is why the solve time goes up fast, from 0.1 s at $N = 100$ to about 4 minutes at $N = 2000$.

For the fluid I used $N = 300$ particles and solved the transport from scratch every frame. (Warm-starting from the previous frame's weights made L-BFGS take a bad first step, so starting cold each frame was both simpler and more stable.) At this size one frame takes about a second, and the 2000-frame run in Figure 3 took around half an hour.

Conclusion

The project covers all the mandatory parts of the three labs:

- Lab 6: Voronoi diagrams by Sutherland–Hodgman bisector clipping, with a parallel $O(N^2)$ build.
- Lab 7: semi-discrete optimal transport, i.e. Laguerre cells, the analytic area/integral/gradient, and L-BFGS on the weights so that every cell reaches its target area.
- Lab 8: a Gallouet–Mérigot fluid simulation built on partial optimal transport with an air variable and per-particle disk clipping, with gravity, the incompressibility spring and wall collisions.

Beyond the required parts I added optional viscosity and softer (inelastic) bounces behind one flag, so the fluid loses energy and settles instead of bouncing forever.

The thing I learned the most from was the volume bug. A small geometry corner case (a zero-radius disk that clipped nothing instead of everything) quietly broke the physics and only showed up as a sudden blow-up about a hundred frames in. Once that one line was fixed the simulation was stable, and the optimal transport step kept the fluid incompressible for thousands of frames.